



CodeEnhancer: LLM-generated Python code enhancement through SAST integration and fine-tuning

Jongmin Lee ^{a,*,}, Khang Mai ^{a,b,c,}, Nakul Ghatge ^{d,}, Tomohiko Yagyu ^{d,}, Razvan Beuran ^{a,}, Yasuo Tan ^{a,}

^a Japan Advanced Institute of Science and Technology, 1-1 Asahidai, Nomi, Ishikawa, 923-1211, Japan

^b University of Information Technology, Ho Chi Minh City, Viet Nam

^c Vietnam National University, Ho Chi Minh City, Viet Nam

^d NEC Corporation, Shiba 5-7-1, Minato-ku, 108-8001, Tokyo, Japan

ARTICLE INFO

Keywords:

Large Language Models (LLMs)
Syntax checking
Vulnerability detection
Functional correctness
Static application security testing
LLM fine-tuning

ABSTRACT

Despite the rapid adoption of Large Language Models (LLMs) for automatic code generation, their output often exhibits syntax errors, security vulnerabilities, and functional inconsistencies. To address these issues, we present CodeEnhancer, a two-stage framework that tightly integrates LLMs with static application security testing (SAST) tools and targeted fine-tuning. The goal is to produce more secure and functionally correct Python code. In the first stage, our iterative validation pipeline couples LLM-generated code with tools such as Pylint and Bandit. These tools automatically identify and remediate issues through structured feedback loops. When applied to the GPT-4o model, this process eliminated 82.8% of the initial vulnerabilities and resolved all the detected functional correctness issues when tested on the LLMSecEval dataset. In the second stage, we fine-tune the LLMs using two types of secure code examples: expert-written samples and code refined by our framework. Comparative experiments demonstrate that the framework-tuned model outperforms the baseline and expert-tuned models. The framework-tuned model generates only 18.4% vulnerable code snippets on the LLMSecEval dataset, whereas the baseline and expert-tuned models produce 43.6% and 54.7% vulnerable code snippets, respectively. The framework-tuned model reduces final vulnerability rates to 6.7% on LLMSecEval and 3.5% on the SecurityEval dataset. Our results highlight the synergistic effect of integrating static analysis with feedback-informed fine-tuning. They also reveal limitations in current evaluation metrics and dataset representativeness. These findings suggest a scalable, robust approach to achieving more secure, trustworthy, and practical AI-assisted code generation.

1. Introduction

The rapid advancement of Large Language Models (LLMs), such as GPT-4 [1], Code Llama [2], and Codex [3], is revolutionizing software development. For example, via GitHub Copilot, LLMs are integrated into daily programming workflows. LLMs generate code from natural-language descriptions, accelerating development cycles and automating complex tasks [1,3]. This offers substantial potential for boosting developer productivity and efficiency.

However, this transformative potential is tempered by critical concerns regarding the quality and reliability of the code generated by LLMs, particularly with regard to security vulnerabilities. While these models can produce functional code snippets, they often generate code with syntactic errors, stylistic inconsistencies, and critical security vulnerabilities that can lead to severe real-world consequences [4,5].

Empirical studies have confirmed that code generated by popular LLMs often contains weaknesses listed in the MITRE Common Weakness Enumeration (CWE) Top 25. These weaknesses include command injection (CWE-78), SQL injection (CWE-89), path traversal (CWE-22), and insecure use of cryptography (CWE-327) [6,7].

Furthermore, ensuring that the generated code precisely implements the functionality specified in the natural language prompt, a property often referred to as functional correctness or prompt adherence, remains a notable hurdle. Although execution-based evaluations on standard coding benchmarks [3] can verify correctness against pre-defined test cases, they may overlook subtle deviations in nuanced free-form prompts. Previous research has demonstrated that LLMs can generate code that seems plausible or even passes certain tests yet contains logical errors or fails to align with the user's intended behavior,

* Corresponding author.

E-mail addresses: jongmin@jaist.ac.jp (J. Lee), khangmt@uit.edu.vn (K. Mai), te.nak14@nec.com (N. Ghatge), yagyu@nec.com (T. Yagyu), ytan@jaist.ac.jp (Y. Tan).

<https://doi.org/10.1016/j.knosys.2026.115925>

Received 17 July 2025; Received in revised form 26 March 2026; Accepted 1 April 2026

Available online 2 April 2026

0950-7051/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

resulting in functional bugs [6]. Automatically evaluating this alignment is difficult. Metrics designed primarily for code similarity, such as CodeBLEU [8], CodeBERTScore [9] and CodeScore [10], measure structural or semantic resemblance to reference code. However, they cannot inherently verify behavioral equivalence or guarantee that the code accurately fulfills the prompt's specific functional goals. This gap highlights the ongoing challenge of automatically validating the true functional correctness and prompt fidelity of LLM-generated code.

To comprehensively address these multifaceted issues, this paper proposes a novel two-stage framework named CodeEnhancer that improves the security, correctness, and quality of LLM-generated Python code. The framework integrates automated validation with targeted model improvement through fine-tuning. As a contribution to the research community, we provide the full implementation of the CodeEnhancer framework on GitHub.¹

The *first stage* of CodeEnhancer consists of an iterative validation and refinement pipeline. This pipeline couples the LLM's code generation capabilities with established Static Application Security Testing (SAST) tools. Specifically, it uses Pylint [11] to check for syntax issues, and Bandit [12] to detect security vulnerabilities. When the LLM generates an initial code snippet based on a natural language prompt, these SAST tools are used to analyze it. Any detected issues trigger a structured feedback mechanism that provides the LLM with specific details, such as the type, location, and severity of the issue, which are necessary to revise the code. The generate-validate-refine loop then iterates, progressively improving the code until it passes all checks or reaches a predefined iteration limit.

While this iterative validation pipeline is effective for post-generation correction, it has limitations when relied on solely. The process can be computationally intensive and may struggle with complex vulnerabilities that require a deeper understanding of security. Additionally, it does not improve the LLM's inherent ability to proactively generate secure code. The LLM remains reactive, depending entirely on external tool feedback.

Therefore, the *second stage* of our framework introduces a targeted fine-tuning methodology to enhance the LLM's intrinsic security awareness and code generation capabilities. This stage adapts the base LLM (GPT-4o) using specialized datasets containing secure coding examples. In particular, fine-tuning is done with one of the following: (1) an existing dataset comprising expert-written secure code examples, representing best practices and an idealized security standard [13], and (2) a novel dataset curated from the framework-refined code outputs produced by the first stage (the validation pipeline) of our own framework. Thus, we can explore whether LLMs benefit more from learning idealized secure patterns or from learning the practical process of transforming potentially flawed code into secure, validated code via automated feedback loops. We hypothesize that fine-tuning on framework-refined data will better prepare the LLM to handle real-world coding scenarios and generate more robust, secure code from the beginning.

The synergy between these two stages is crucial. The fine-tuned LLM produces better initial code, reducing the workload and the number of iterations required in the validation pipeline. At the same time, the validation pipeline can generate better, more refined code examples that could be used for further stage of model improvement, creating a virtuous cycle.

The primary contributions of this paper are:

1. The design and implementation of a comprehensive, two-stage CodeEnhancer framework integrating an iterative LLM-SAST validation pipeline and a subsequent targeted LLM fine-tuning methodology to enhance Python code security.
2. A rigorous evaluation demonstrating the framework's effectiveness using the LLMSecEval benchmark [13] to validate the first stage and the LLMSecEval and SecurityEval benchmarks [14] to assess the performance of the fine-tuned models in the second stage. In particular, the evaluation compares the effectiveness of fine-tuning with datasets derived from expert-written secure examples and those generated by the automated refinement process. The results highlight the benefits of the integrated approach and the superiority of leveraging framework-refined data for fine-tuning. Notably, this approach outperformed expert-written examples in terms of model performance and proved to be more cost-effective.
3. An analysis provides insights into the synergistic benefits of combining automated validation with model adaptation. This contributes to the development of more trustworthy and secure AI-assisted coding practices.

The remainder of this paper is structured as follows: Section 2 provides a background and motivation for the research, and Section 3 reviews related work in LLM-based code generation, security vulnerability detection, static analysis, and fine-tuning approaches. Section 4 details the proposed methodology, elaborating on both the real-time validation pipeline and the fine-tuning extension. Section 5 outlines the experiment setup, including datasets, models, and evaluation metrics. Section 6 presents and analyzes the experimental results, comparing the performance of the baseline and fine-tuned models. Section 7 discusses the limitations of the current work and suggests directions for future research. Finally, Section 8 concludes the paper by summarizing the key findings and contributions.

2. Background and motivation

The advent of LLMs is a significant milestone in artificial intelligence that has had a profound impact on various domains, including software engineering. Models such as Mistral [15], GPT-4 [1], and DeepSeek-R1 [16] are trained on vast datasets containing billions of lines of code and natural language text. This extensive training enables these models to discern complex patterns, syntax, and common programming idioms across multiple languages. Consequently, these models can generate code directly from natural language prompts, offering substantial potential to accelerate software development [17]. Integrated tools, such as GitHub Copilot, leverage these capabilities by automating routine coding tasks, allowing developers to focus on higher-level system design and problem-solving and boosting productivity [4].

Alongside the rise of AI-driven code generation, Static Application Security Testing (SAST) has become integral to modern secure software development practices. SAST tools analyze application source code, bytecode, or binary code without executing it, aiming to detect potential bugs, style inconsistencies, and security vulnerabilities early in the development lifecycle [18]. SAST tools employ techniques such as pattern matching, data flow analysis, and control flow analysis to identify deviations from coding standards, such as using Pylint [11] to check for compliance with Python's PEP 8, and common security pitfalls such as those cataloged by OWASP or MITRE. Security-focused SAST tools like Bandit [12] for Python or the more general-purpose CodeQL [19] specifically target known vulnerability patterns, including SQL injection, cross-site scripting (XSS), hardcoded secrets, and insecure API usage. Early detection through SAST is crucial for reducing the cost and effort required for remediation compared to finding flaws later in testing or after deployment.

Despite their impressive code generation capabilities, LLMs are not infallible. A growing body of evidence highlights significant concerns about the quality and security of the code LLMs produce [4–6]. LLM-generated code frequently contains:

¹ <https://github.com/cyb3rlab/CodeEnhancer>

1. **Syntax Errors:** Recent work [20] has revealed that syntax errors account for most faults in LLM-generated Verilog code. Addressing these errors alone achieved a 98.5% fix rate using retrieval-based prompting in the study, highlighting syntax correction as a critical step toward improving the reliability of machine-generated code.
2. **Security Vulnerabilities:** LLMs often replicate insecure coding patterns that are present in their vast training data, generating code susceptible to exploits such as command injection, path traversal, and improper cryptographic function handling [6,7].
3. **Functional Correctness Issues:** Ensuring that the generated code accurately reflects the user's intent, as specified in the prompt, remains a critical challenge [6]. LLMs may misinterpret requirements, overlook edge cases, or produce code with subtle logical flaws, leading to incorrect behavior even if it is syntactically valid [21].

These shortcomings necessitate robust mechanisms to validate and improve LLM-generated code before integrating it into software projects. However, relying solely on LLMs to iteratively fix or refine their own code often yields suboptimal results. This approach suffers from a form of model bias, wherein the same reasoning flaws that led to the initial issues may persist or compound during self-revision. A recent study [22] has demonstrated that LLMs frequently struggle to detect or correct nuanced logical or security flaws in their own outputs, especially in the absence of structured external feedback. Previous research found that LLMs often fail to self-correct reasoning errors without external input, and in some cases, their performance degrades after self-correction. Similarly, another study highlighted that self-correction without external guidance can lead to overfitting to syntactic patterns, which limits the model's ability to ensure functional correctness or robust security guarantees [23].

Other researchers also have shown that fine-tuning large language models on carefully constructed code datasets can improve their ability to generate more secure and functional code. Training on examples that reflect secure coding practices reduces the likelihood of introducing vulnerabilities while improving the syntactic and semantic quality of the outputs. These findings suggest that fine-tuning is an effective mechanism for aligning LLMs with practical coding standards and expectations in real-world development settings [24,25].

Therefore, this paper addresses the limitations of relying solely on LLM code generation, post-hoc SAST analysis, or isolated fine-tuning efforts. There is a clear need for a more holistic approach. We propose and evaluate a comprehensive, integrated framework that combines the strengths of LLMs and SAST tools synergistically. Our framework establishes a virtuous cycle in which an iterative validation pipeline integrates SAST feedback to refine generated code. Simultaneously, a targeted fine-tuning methodology leverages these refined outputs to enhance the LLM's ability to generate secure, functionally correct code from the outset. The overarching goal is to create a more robust and trustworthy pathway for leveraging LLMs in software development by addressing syntax, security, and functional correctness in a unified, multi-stage process.

3. Related work

Research efforts to improve the quality and security of LLM-generated code span several areas, including direct code generation improvements, automated code repair, the use of SAST tools, specific evaluation methodologies, and fine-tuning techniques. Table 1 provides a comparative overview of representative approaches based on their primary focus and the specific aspects of code quality they address, contextualizing our proposed integrated framework.

3.1. LLM-based frameworks for code generation, validation, and repair

Recent advances in LLM-based code generation have led to a diverse array of frameworks that aim to improve the quality, correctness, and security of generated code. As summarized in Table 1, these approaches differ in their primary focus, tool integration, and treatment of validation or repair tasks.

Early work by Chen et al. [3] introduced Codex, a fine-tuned GPT-3 model trained on code data, which was demonstrated to be capable of generating functional code from natural language prompts. However, Codex did not explicitly address code repair or security validation, despite achieving high pass rates on benchmark tasks such as HumanEval.

In contrast, Alrashedy et al. [26] presented a framework that integrates LLMs with SAST tools to automatically patch security vulnerabilities in source code. Their system leverages SAST tool output to generate prompts for LLM-based repair and is one of the earliest pipelines to combine static analysis and generation in an automated feedback loop.

Pearce et al. [27] further investigated the zero-shot repair capabilities of Codex on vulnerable code snippets. Although no fine-tuning or external validation tools were used, the study revealed the potential and limitations of prompting alone in addressing security-related issues.

More recent work has shifted toward improving the intrinsic ability of LLMs to produce secure code through fine-tuning. For example, Li et al. [24] conducted an empirical study on fine-tuning models using security patch commits and demonstrated measurable improvements in vulnerability reduction. Similarly, Weyssow et al. [25] explored parameter-efficient fine-tuning (PEFT) methods for code generation, however, their study focused more on syntax and structural quality than on vulnerability mitigation.

Separately, Keltek et al. [28] proposed LSAST, a hybrid system in which SAST tools, such as Bearer, are paired with LLMs to enhance vulnerability detection coverage. Although LSAST is not explicitly aimed at code generation or repair, it exemplifies the complementary use of LLMs for security analysis and code validation.

Building on these advancements, research published in 2024 and 2025 has scrutinized the efficacy of autonomous self-correction in LLMs. For example, Olausson et al. [30] demonstrated that relying solely on an LLM's intrinsic self-repair capabilities without robust external feedback often yields diminishing returns or introduces new errors. To overcome this limitation, the field is rapidly shifting toward iterative optimization frameworks. Ridnik et al. [29] introduced flow engineering approaches that use test-driven validation to refine code iteratively, while Kumar et al. [31] explored reinforcement learning methods to train models specifically for self-correction.

Unlike prior studies that intentionally prompt the model to generate insecure code by using artificial inputs, focus on isolated aspects such as generation and repair, or struggle with the limitations of intrinsic self-correction, our framework integrates all these components into a unified pipeline that employs natural, real-world prompts. By combining iterative static validation with fine-tuning on both expert-written and framework-refined code, our approach systematically bypasses the unreliability of autonomous self-repair highlighted in recent literature. It addresses syntax errors, functional correctness, and security vulnerabilities cohesively, firmly positioning our work within the rapidly evolving domain of automated, feedback-driven code optimization.

3.2. Code evaluation using SAST tools

SAST tools are standard instruments for security assessment. For Python, Bandit [12] targets common security flaws, and Pylint [11] checks for code smells, style issues, and syntax errors. More sophisticated tools like GitHub's CodeQL adopt semantic analysis to detect complex vulnerabilities across multiple languages [19]. Researchers

Table 1
Comparison of related research works and CodeEnhancer.

	Chen et al. [3]	Alrashedy et al. [26]	Pearce et al. [27]	Li et al. [24]	Weyssow et al. [25]	Keltek et al. [28]	Ridnik et al. [29]	Olausson et al. [30]	Kumar et al. [31]	CodeEnhancer
Feature/Capability										
Code Generation	✓	✓	-	✓	-	-	✓	✓	✓	✓
Code Validation & Repair	-	✓	✓	-	✓	✓	✓	✓	✓	✓
Uses LLM	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Uses SAST Tools	-	✓	✓	-	-	✓	-	-	-	✓
Employs Fine-tuning	✓	-	-	✓	✓	-	-	-	✓	✓
Addresses Syntax Errors	-	-	-	-	-	-	-	-	-	✓
Addresses Security Vulnerabilities	-	✓	✓	✓	-	✓	-	-	-	✓
Addresses Functional Correctness	✓	-	-	-	✓	-	✓	✓	✓	✓

✓: Supported/Primary Focus; (~): Partially Supported/Secondary Focus/Implicit; -: Not Supported/Not Within Scope.

utilize these tools extensively to evaluate the security posture of LLM-generated code [32]. However, studies caution against relying on a single tool, as their detection capabilities vary across vulnerability types [33]. Even advanced tools like CodeQL can miss vulnerabilities. Enterprise tools, such as SonarQube [34] and Checkmarx [35] also exist. These tools offer broader capabilities and integrations, but they come with different complexities and costs.

3.3. Functional correctness evaluation

Validating that generated code behaves according to the intention expressed in the prompt is crucial yet challenging. Metrics such as CodeBLEU [8] and CodeBERTScore [9] measure code similarity by directly comparing the reference code and the target code. Execution-based testing using benchmarks such as HumanEval [3] or The Mostly Basic Programming Problems (MBPP) [36] measures functional correctness by checking pass rates against predefined unit tests [37]. While valuable, this approach might not fully cover the prompt's specification. To address this issue, newer metrics aim for better alignment without relying solely on reference code. CodeScore [10] trains a model to predict execution outcomes. ICE-Score [38] uses an LLM itself to assess LLM-generated code against the given input prompt. SBC-Score [39] employs a novel reverse-generation technique, inferring requirements from the generated code and comparing them back to the original prompt using semantic similarity, lexical overlap, and completeness measures. Our framework incorporates a functional validation step using LLM reasoning based on prompt-derived specifications.

4. CodeEnhancer methodology

CodeEnhancer improves the security and quality of LLM-generated Python code with a two-stage approach: (1) a code generation & validation and refinement pipeline that integrates LLMs with SAST tools, and (2) a fine-tuning methodology that improves the LLM's secure coding capabilities. Fig. 1 provides an overview of this integrated architecture.

4.1. Stage 1: Code generation & validation and refinement pipeline

The first stage focuses on generating code and iteratively improving it based on immediate feedback from SAST tools. This pipeline consists of several key steps designed to address syntactic, security, and functional correctness progressively, thus being in alignment with a structured quality assurance process.

4.1.1. Code generation with structured prompt template

LLMs often produce syntactically correct yet structurally inconsistent or incomplete code when prompted with free-form instructions. To address this, our framework adopts a structured prompt template that demonstrates the desired output format and enforces a consistent generation template across diverse tasks.

The process begins with a natural language prompt provided by the user that describes the desired code functionality. To enhance clarity and facilitate subsequent validation, we employ a structured prompt template. This template instructs the LLM to generate functional code and a detailed docstring that restates the user's request, specifies the code's purpose, and describes the steps to achieve the intended functionality. The LLM then processes this structured input to generate an initial Python code snippet with an embedded docstring that serves as a functional specification.

The core idea is to guide the LLM to include a module-level docstring with three required fields at the top of each Python file.

1. **Input Prompt** — a restatement of the original user request.
2. **Intention** — the purpose or goal of the function.
3. **Functionality** — a precise description of how the function achieves that goal.

This docstring improves the code and enables automated downstream evaluation. For example, our functional correctness validation leverages LLM reasoning to compare the functionality described in the docstring with the actual code logic.

4.1.2. Multi-faceted code validation

The Code Validation component iteratively improves the quality and security of LLM-generated code by detecting and correcting errors across multiple dimensions. This process integrates LLMs with SAST tools using three validation components, as follows:

1. **Syntax Error Validation:** The framework incorporates Pylint to validate the syntactic correctness and structure of the generated code. This includes identifying missing or misplaced elements such as parentheses, indentation inconsistencies and improper use of reserved keywords. Identified issues are logged, and corrective prompts are sent to the LLM, which iteratively refines the code to align with these standards. This ensures that the generated code is both executable and adheres to basic syntactic conventions. Fig. 2 provides an example of a syntax issue detected by Pylint, illustrating the type of feedback used for iterative correction.
2. **Vulnerability Validation:** To ensure security, the framework integrates Bandit, a SAST tool specifically designed for Python. Bandit scans the code for vulnerabilities, such as command

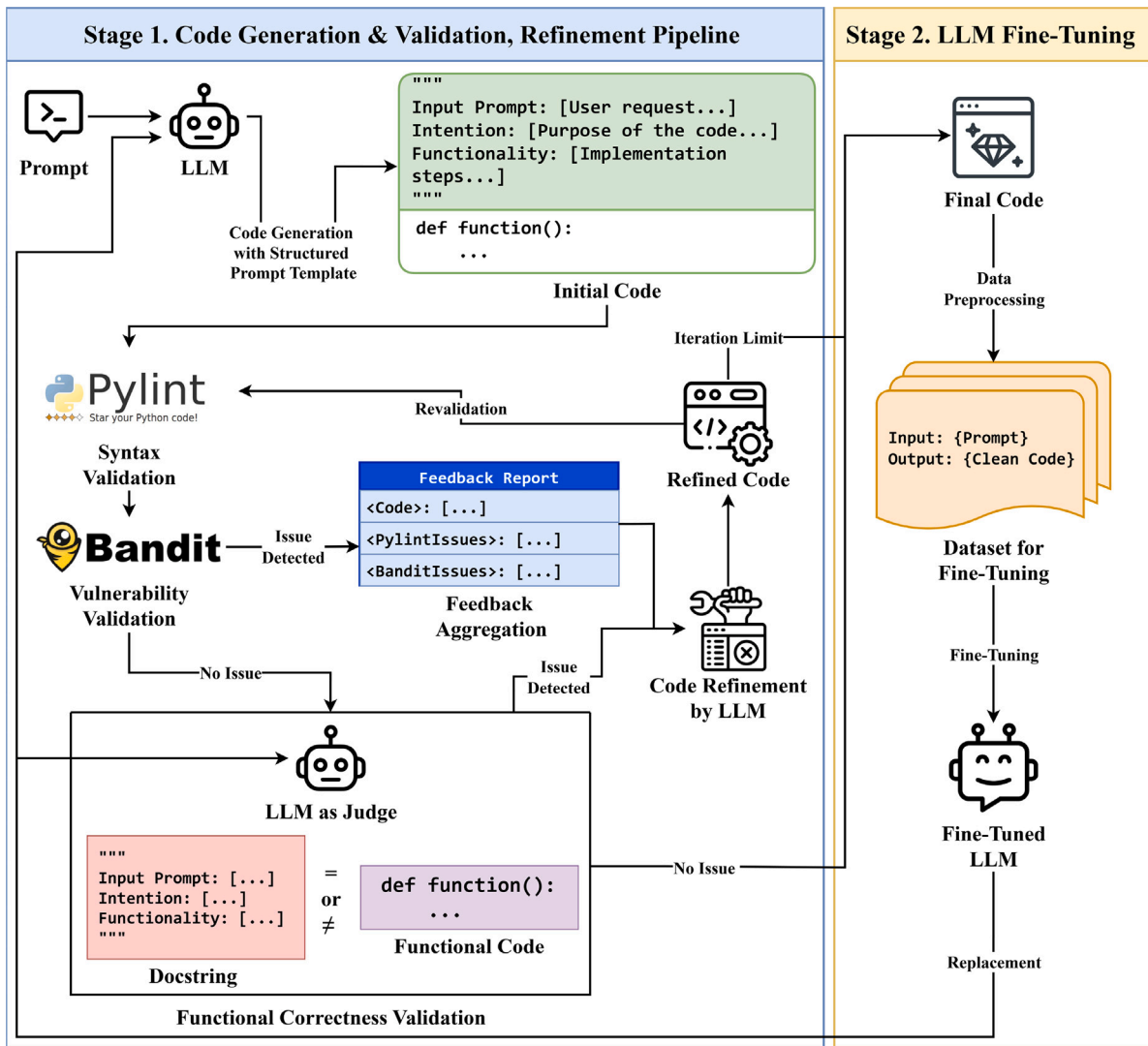


Fig. 1. Overall architecture of CodeEnhancer, illustrating the relationship between the code generation & validation refinement pipeline (Stage 1) and the fine-tuning methodology (Stage 2).

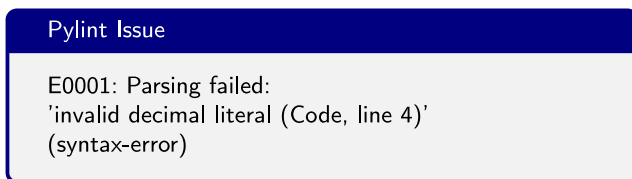


Fig. 2. Example of a code syntax issue detected via Pylint.

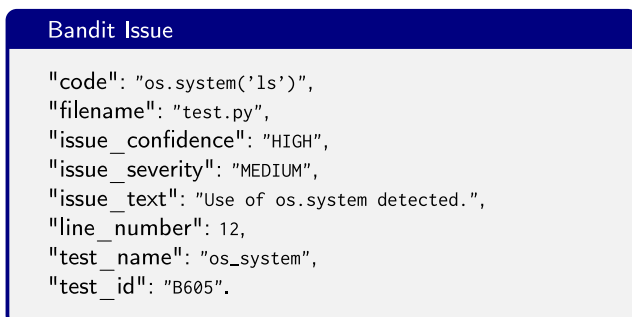


Fig. 3. Example of a code vulnerability issue detected via Bandit.

injections, insecure data storage, or improper use of sensitive configurations such as hardcoded passwords. Each detected vulnerability is linked to a specific CWE identifier for precise categorization and tracking. When vulnerabilities are detected, they are logged with detailed descriptions, including CWE identifiers and severity levels. The framework then prompts the LLM to revise the code accordingly, aiming to eliminate the detected issues while preserving the original functionality. By leveraging the functionality of Bandit, the system proactively reduces the risk of having security vulnerabilities. Fig. 3 shows an example of a security issue identified by Bandit, which serves as a basis for the targeted code correction through iterative refinement.

3. **Functional Correctness Validation:** Ensuring that the code performs as intended is crucial for practical deployment. For this purpose, the LLM compares the logic of the generated code against its associated docstring. The docstring serves as a functional specification, and the LLM identifies discrepancies by parsing the described behavior and matching it with the implemented logic. To avoid conflicts between functional correctness and security, the validation does not enforce exact implementation details. Rather, it considers the code correct as long as it achieves the same intended behavior, even if different libraries or algorithms are used to improve security or performance. For instance, if the docstring indicates that the function

retrieves user data from a database, but the implemented code contains only a placeholder and no actual database queries, the framework will flag this discrepancy during validation. Then, the LLM autonomously modifies the code to align it with the specified requirements. This step validates functional correctness and improves adherence to the intended design and usability standards.

4.1.3. Feedback aggregation and formatting

If any issues are detected during the validation stage, the findings from Pylint, Bandit, and the functional correctness check are collected and formatted into a structured feedback report. To ensure clarity, this feedback report is formatted as a structured natural language prompt using XML-like markup tags, such as `<Code>`, `<PylintIssues>`. This report typically includes:

1. The specific tool that flagged the issue.
2. The code snippet containing the issue.
3. An explanation of the issue, as provided by the tool or the LLM.

This structured markup explicitly separates the current code implementation from the raw error messages. This provides the LLM with unambiguous context and strict formatting rules. This approach is crucial for effectively guiding the LLM in the refinement step.

4.1.4. Iterative code refinement

If any issues are identified, the aggregated feedback is sent back to the LLM along with the original prompt and the current (flawed) code version. The LLM is then instructed to revise the code to specifically address the reported issues while preserving the original docstring and intended functionality. This generate-validate-refine cycle repeats iteratively. The loop terminates under one of the following conditions:

1. All the validation steps (syntax, security, and functional correctness) are passed successfully.
2. A predefined maximum number of iterations is reached.

This iterative process enables the framework to improve the code progressively. The final output of this stage is either a validated code snippet or the best version achieved within the iteration limit.

In our experiments, the maximum number of iterations was empirically set to five. This is because preliminary tests showed that most correctable syntax errors and security vulnerabilities were resolved within three to four iterations. Our experiment results that are presented later in the paper further support this statement, since we observed no changes in the cumulative percentage of secure code snippets between four and five iterations (see Fig. 8). Consequently, setting the iteration limit to five is enough to accommodate functional corrections, while ensuring the LLM does not enter an infinite loop. This decision also ensures the framework's overall efficiency and minimizes LLM API costs.

4.2. Stage 2: LLM fine-tuning

Although the validation pipeline effectively corrects issues, the second stage of the framework proactively enhances the LLM's ability to generate secure code snippets through fine-tuning. This process involves dataset preparation, model training, and integration.

4.2.1. Dataset preparation and model training

A key innovation of our framework is that the fine-tuning dataset is constructed without expert intervention using the outputs from Stage 1. Thus, rather than relying on manual dataset construction, our framework automatically generates the fine-tuning dataset by pairing each input prompt with the code produced after a comprehensive validation and refinement pipeline. Only code that passes all automated checks is included in the final dataset for supervised fine-tuning. This approach

LLMSecEval Prompt Example

Generate Python code for the following:
The code reads a value from the command line and adds 1000 to it.

Fig. 4. Example of a natural language prompt from the LLMSecEval dataset.

greatly reduces the manual effort typically required, and also improves the quality and reproducibility of dataset preparation.

The general supervised fine-tuning approach utilizes prompt and code pairs. To evaluate different fine-tuning strategies, we constructed two distinct datasets with input prompts from the LLMSecEval dataset. These prompts were paired with two types of target source code: (1) framework-refined code, which is automatically generated optimized code produced after our Stage 1 validation pipeline with docstrings excluded, and (2) expert-written secure code, consisting of manually written secure code examples provided in the dataset.

4.2.2. Integration and evaluation

The fine-tuned models can then be integrated back into Stage 1, replacing the baseline LLM. We hypothesize that the fine-tuned models will generate higher-quality initial code, improving the efficiency and effectiveness of the overall framework. The evaluation of the comparative performance of the baseline and fine-tuned models will be presented in Section 6.

5. Experiment setup

This section introduces the experiment setup, including the datasets and LLMs used, as well as the evaluation metrics. Specifically, we evaluated Stage 1 of the framework using Experiment #1, and Stage 2 using Experiment #2.

5.1. Datasets

In our experiments, we utilized two datasets, which are described below.

LLMSecEval. The LLMSecEval dataset [13] is used in both experiments. This dataset is specifically designed to evaluate the security and robustness of LLM-generated code. This dataset is based on the MITRE CWE Top 25 Most Dangerous Software Weaknesses and covers 21 distinct CWE categories. These categories include common issues, such as buffer overflows (CWE-120), command injection (CWE-77), and improper input validation (CWE-20). The dataset contains 150 carefully designed natural language prompts, each of which is crafted to induce specific coding vulnerabilities in Python when processed by LLMs. These prompts simulate real-world development scenarios in which insecure code may be generated inadvertently.

Each prompt is accompanied by multiple secure reference implementations written by domain experts to provide ground truth examples as a reference. The dataset enables precise benchmarking of code quality in terms of vulnerability presence, syntax correctness, and functional behavior. This makes it a comprehensive resource for evaluating security-oriented LLM applications (see Fig. 4).

SecurityEval. For Experiment #2, we used the SecurityEval dataset [14], which covers a broader and more diverse set of security-sensitive tasks. Unlike LLMSecEval, SecurityEval targets various vulnerability types beyond the MITRE Top 25 and is designed to challenge the generalization capabilities of LLMs. The dataset consists of 121 natural language prompts that span 69 distinct CWE categories, offering wide coverage across both common and less frequently addressed vulnerabilities. We adopted SecurityEval for Experiment #2 because the models

SecurityEval Prompt Example

```
import yaml
def yaml_load(filename):
    '''
    use the filename variable to open a file,
    load its contents into a variable with yaml,
    and return it
    '''
```

Fig. 5. Example of a natural language prompt from the SecurityEval dataset.

being evaluated were fine-tuned based on the output of Experiment #1, and a new dataset was necessary to avoid overfitting and assess generalization performance (see Fig. 5).

5.2. LLM configurations

We used different LLM models in each experiment, as explained next.

Baseline Model (Experiment #1): In this research, we adopted GPT-4o for both code generation and validation tasks. Although the LLM-SecEval dataset was originally evaluated using a GPT-3 based model (code-davinci-002, now discontinued), we upgraded to GPT-4o to leverage its performance and versatility. Its contextual understanding and iterative refinement capabilities make it exceptionally well-suited for our framework, enabling the generation of secure, functional, and high-quality Python code that robustly addresses diverse vulnerabilities.

Baseline and Fine-Tuned Models (Experiment #2): In Experiment #2, we evaluated three model variants:

1. **Baseline:** A basic GPT-4o model, used as a baseline for comparison.
2. **Fine-Tuned Model Using Expert Examples:** GPT-4o that has been fine-tuned using expert-written secure code examples from the LLMSecEval dataset.
3. **Fine-Tuned Model Using Framework Output:** GPT-4o is fine-tuned using the framework-refined code produced through our validation pipeline in Experiment #1 using LLMSecEval.

We executed our full framework five times for each model, producing five sets of code outputs before and after refinement.

5.3. SAST tool version

To ensure the reproducibility of our results, we report here the version of each SAST tool used in our experiments. Thus, Pylint v3.3.7 was employed for syntax validation, and Bandit v1.8.3 for security vulnerability detection throughout all our experiments. We note that these were the latest stable releases available for the two tools at the time the experiments were conducted.

5.4. Evaluation metrics

We used multiple evaluation criteria to assess the models comprehensively. In all cases, the iterative refinement process was performed with a maximum limit of five iterations.

Syntax Errors: We adopted Pylint to detect syntax errors in the generated code, not for PEP-8 style checks.

Vulnerabilities: We chose Bandit to identify security vulnerabilities according to CWE categories. We focus on the number of detected vulnerabilities and their resolution rates after refinement.

Functional Correctness: First, we manually evaluated functional correctness by inspecting whether the code accurately implements the intended functionality described in the prompt and the embedded docstring.

Additionally, we evaluated the quality of the generated code using three advanced metrics:

1. **CodeBLEU:** This metric measures syntactic and semantic similarity based on n-gram matching, AST structure similarity, and data flow consistency [8].
2. **CodeBERTScore:** It uses transformer-based contextual embeddings to assess semantic alignment between the generated code and the Ref. [9].
3. **ICE-SCORE:** It evaluates how well the generated code complies with the given instructions and its likelihood of being executable, using an LLM-based review system. It uses two separate 0–4 scales to independently assess usefulness (how helpful the code is for solving the task) and functional correctness (how accurately the code implements the intended behavior) independently [38].

6. Experiment results and analysis

This section presents the results of our experiments, analyzing the performance of the three model configurations in generating secure Python code. We examined vulnerability detection and resolution, code quality metrics, and the effectiveness of our fine-tuning methodology.

6.1. Experiment #1: Code generation & validation and refinement results

The Experiment #1 was evaluated using 150 Python code snippets, each of which was generated using prompts from the LLMSecEval dataset. These prompts are designed to test an LLM's ability to handle security-sensitive tasks aligned with CWE categories.

6.1.1. Syntax error validation

No syntax errors were detected in any of the 150 Python code snippets generated during Experiment #1 of our framework evaluation. While preliminary experiments using earlier versions of LLMs had occasionally resulted in syntax issues such as indentation errors or missing delimiters, all code generated using the GPT-4o model in this study passed syntax validation on the first attempt. This suggests that GPT-4o is capable of producing syntactically correct Python code when guided by a well-structured prompt and docstring.

6.1.2. Vulnerability validation

During the vulnerability validation phase, a total of 64 code snippets with vulnerabilities were detected in the initial code generated from the LLMSecEval dataset. After the iterative refinement process, this number decreased to 11, corresponding to an 82.8% resolution rate, as shown in Fig. 6(a). This result highlights the effectiveness of our validation framework in addressing the majority of security issues through automated feedback and code correction.

Our analysis showed that the unresolved CWE-78 vulnerabilities after iterative refinement were primarily caused by the use of the subprocess module in Python. This module is often used to execute shell commands and introduces risks when user inputs are directly concatenated into commands. Although Bandit flagged the use of the subprocess module with warnings, the framework failed to effectively address these vulnerabilities effectively by enforcing strict validation of user inputs. The lack of input sanitization allowed these security issues to persist, underscoring the critical importance of incorporating comprehensive input validation mechanisms to mitigate risks associated with shell command execution.

Bandit's analysis of secure code examples written by experts from LLMSecEval revealed similar CWE-78 issues when the subprocess module was used. In these instances, the same errors were flagged in the

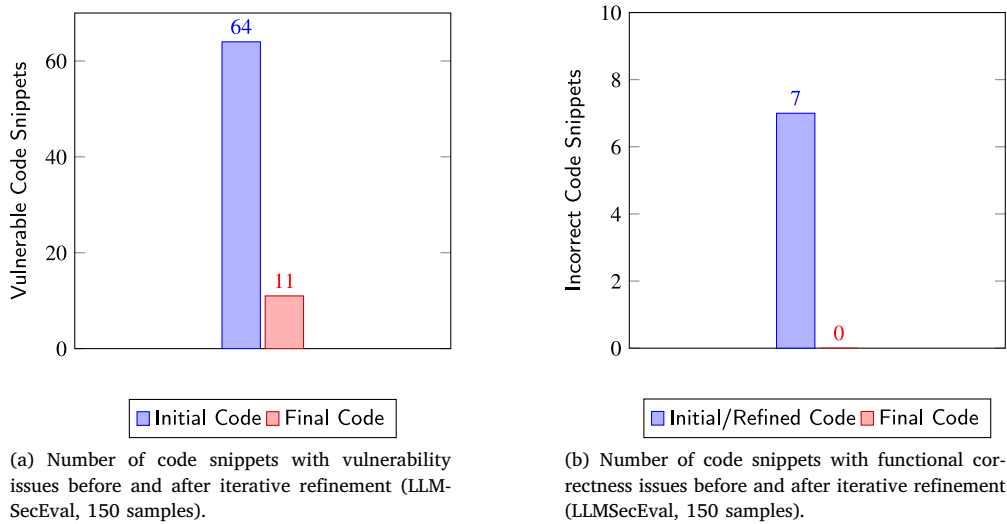


Fig. 6. Experiment #1: Comparison of the number of code snippets with vulnerability and functional correctness issues before and after iterative refinement when using the LLMSecEval dataset.

expert-designed secure code examples as in the framework's generated code. Upon further inspection, it was observed that the refined code snippets and secure examples often employed similar strategies to mitigate these vulnerabilities. These approaches typically included isolating user inputs or implementing explicit whitelisting mechanisms to reduce risks. While these refinements improved the overall security posture, they did not eliminate the vulnerabilities entirely, which highlights the limitations of current strategies in addressing CWE-78 issues involving the subprocess module.

6.1.3. Functional correctness validation

As shown in Fig. 6(b), among the 150 code snippets generated and evaluated, the LLM itself identified 7 cases with potential functional correctness issues during the iterative validation process due to inconsistencies between the implemented logic and the corresponding docstring. This represents 4.7% of the total. This self-diagnosis mechanism demonstrates the model's ability to reason about its own outputs through prompt-guided validation. Notably, codes with issues were successfully resolved through iterative refinement within several feedback iterations. This demonstrates that the framework effectively detects subtle discrepancies and autonomously resolves most of them without external feedback.

The types of issues detected by the LLM included missing essential module imports, the need for improved input handling to enhance functionality, and using more appropriate modules instead of suboptimal choices. In each case, the LLM attempted to revise the implementation to better align with the requirements stated in the docstring. These findings underscore the effectiveness of our approach in identifying and incrementally improving code quality based on explicit functional specifications.

6.1.4. Quantitative results using code-level metrics

Additionally, we conducted a quantitative evaluation using three code-level metrics.

CodeBLEU Evaluation: To evaluate the impact of the code validation & refinement stage further, we compared the CodeBLEU scores of code snippets. We conducted the comparison across three categories: initial code generated before the iterative refinement process, refined code after completing the iterative refinement process, and secure code examples from the LLMSecEval dataset, which served as benchmarks.

As shown in Table 2, the CodeBLEU score improved slightly after the iterative refinement process. A notable increase was observed in

Table 2

Experiment #1: Average CodeBLEU score comparison of secure code examples versus initial/final code snippets (LLMSecEval, 150 samples).

Metric	Initial code	Final code
N-gram Match Score	0.074	0.073
Weighted N-gram Match Score	0.091	0.091
Syntax Match Score	0.418	0.421
DataFlow Match Score	0.366	0.381

the dataflow match metric. When analyzing all 150 code snippets individually, most of the CodeBLEU evaluation metrics improved after the iterative refinement process, though some remained consistent. These results suggest that the refinement process brought the generated code closer to the secure code examples in terms of syntactic and functional correctness. This provides evidence of consistent enhancement throughout the validation and refinement process.

Although the overall CodeBLEU scores were relatively low, this can be attributed to the nature of the secure code examples that were used as benchmarks. Human experts manually implemented these examples, resulting in high-quality reference implementations. In contrast, our framework generates code based on GPT-4o, which may introduce architectural and stylistic differences, including library choices and error-handling patterns. Therefore, it is difficult for framework-generated code to achieve a high overlap with these expert-crafted samples, especially under reference-based metrics such as CodeBLEU that reward superficial form similarity.

Therefore, fine-tuning the model directly on expert-written code that follows different structural conventions than the target model's native generation distribution may limit the model's ability to adapt effectively and may result in unexpected behaviors, such as incomplete outputs or inconsistencies in the generated code.

CodeBERTScore Evaluation: Additionally, we evaluated the effects of the code validation stage on code snippets using CodeBERTScore metrics. This evaluation considered three categories: initial outputs before iterative refinement, outputs after refinement, and secure code examples from the LLMSecEval dataset provided by experts.

Table 3 presents a detailed CodeBERTScore comparison among three pairs: (1) secure, expert-written code versus initial, generated code; (2) secure, expert-written code versus final, refined code; and (3) initial versus final code snippets. The results show that the similarity

Table 3

Experiment #1: Average CodeBERTScore comparison of secure code examples versus initial/final code snippets, and initial versus final code snippets (LLM-SecEval, 150 samples).

Metric	Secure vs. Initial	Secure vs. Final	Initial vs. Final
Precision	0.820	0.816	0.988
Recall	0.821	0.822	0.994
F1	0.820	0.819	0.991
F3	0.820	0.821	0.994

Table 4

Experiment #1: Average ICE-SCORE comparison of initial versus final code snippets (LLM-SecEval, 150 samples).

Metric	Initial code	Final code
Functional Correctness	3.987	3.953
Usefulness	3.960	3.893

between the initial and final code snippets is extremely high, indicating that most changes during refinement are minimal and localized. Comparing both initial and final outputs to secure code examples reveals similar scores, suggesting the refinement process brings marginal improvements in overall semantic similarity to expert-written code. However, the recall and F3 metrics show a slight increase after refinement, indicating a small improvement in alignment with the expert code after the iterative process. These results demonstrate that, while the refinement process enhances certain aspects of code quality, it primarily preserves the original structure, making only minor, targeted corrections rather than large-scale modifications.

Furthermore, a manual review comparing the refined and initial outputs highlighted the framework's ability to incorporate additional functionalities, such as enhanced validation mechanisms, improved robustness, and more comprehensive input handling. Importantly, even after the code validation experiment, the refined code snippets retained a level of functional and semantic consistency comparable to that of expert-designed secure code examples while offering capabilities beyond those included in the examples.

While this finding underscores the strength of LLMs in applying precise and minimal edits during refinement, it also aligns with the limitations discussed in Section 6.1.2. Specifically, the framework's tendency to preserve the original code structure may contribute to its inability to resolve deeply embedded or architectural security issues, such as those observed in unresolved CWE-78 cases. These results highlight a key trade-off in the refinement process: balancing minimal intervention with the need for comprehensive structural transformation when necessary.

ICE-SCORE evaluation: To assess how well each code variant fulfilled the instructional intent conveyed through the input prompts, ICE-SCORE metrics were employed on both the initial and final codes across the full set of 150 code snippets.

As shown in Table 4, both the initial and final codes achieved nearly perfect scores, with only a very slight decrease observed after refinement. This suggests that, although iterative refinement involved additional actions, such as patching vulnerabilities or adding exception handling, these modifications did not adversely affect the code's overall alignment with the original prompt or its usefulness.

Overall, our quantitative results demonstrate that the refinement process addressed security concerns and enhanced robustness without compromising the intended functionality or instructional fidelity of the code outputs.

However, these results also highlight an important limitation: existing evaluation metrics alone cannot fully capture the effectiveness of our framework. While improvements are observable in standard code-level metrics, current quantitative measures may not fully reflect some

nuanced aspects of security, code robustness, and instruction alignment. Therefore, more comprehensive or specialized evaluation metrics are needed to accurately assess the broader benefits and subtleties of our approach.

6.2. Experiment #2: Fine-tuning results

Experiment #2 investigates the effectiveness of fine-tuned LLMs that use different training sources to improve the intrinsic security and quality of generated Python code. While Experiment #1 validated the performance of an unmodified GPT-4o model using the LLM-SecEval dataset, Experiment #2 employs both the LLM-SecEval and SecurityEval datasets for evaluation. The LLM-SecEval dataset continues to play a key role because the outputs generated from it in Experiment #1 also serve as one of the fine-tuning sources.

To mitigate the potential overfitting to the LLM-SecEval dataset, SecurityEval is also utilized. SecurityEval provides a broader and more diverse set of prompts designed to expose security flaws across a wide range of vulnerability types. This ensures that the evaluation results reflect the models' ability to generalize beyond the patterns present in the fine-tuning data. For each dataset, code generation and iterative validation refinement were performed up to five times, in accordance with the methodology of our framework. This approach enables a systematic and fair comparison of model performance across both benchmarks.

Three models are compared: the baseline GPT-4o model, a variant fine-tuned on expert-written secure code examples, and a model fine-tuned on the outputs of our own validation framework. Through this evaluation, we aim to determine which training strategy better equips LLMs to produce secure, functional, and high-quality code without relying solely on post-generation refinement mechanisms.

6.2.1. Syntax error validation

Across the LLM-SecEval and SecurityEval datasets, each prompt was subjected to up to five rounds of iterative validation and refinement after code generation. As discussed in Section 6.1.1, modern LLMs rarely produce syntactically invalid Python code. In our evaluation, only two code snippets with syntax errors were identified among the initial outputs of the SecurityEval dataset. These errors were promptly detected and corrected by the framework's integrated validation pipeline. Incorporating systematic syntax checking as an initial step ensures that our framework preemptively addresses even rare or minor syntactic mistakes, thus enhancing the reliability of downstream security and functional validation. This approach further reduces the likelihood that such issues will propagate in automated code-generation workflows.

6.2.2. Vulnerability validation

The vulnerability validation experiment assessed the number and proportion of code snippets containing security flaws before and after iterative refinement, as detected by Bandit. Fig. 7 summarizes the results for both the LLM-SecEval (150 code snippets) and SecurityEval (121 code snippets) datasets.

For LLM-SecEval, the baseline GPT-4o model initially generated code with security vulnerabilities in an average of 65.4 out of 150 snippets (43.6%), while the expert-tuned model produced 82.0 vulnerable snippets (54.7%). The framework-tuned model produced an average of only 27.6 (18.4%) vulnerable code snippets. After five refinement cycles, these numbers decreased substantially: the baseline GPT-4o model averaged 11.4 (7.6%) residual vulnerable code snippets; the expert-tuned model averaged 22.0 (14.7%) residual vulnerable code snippets; and the framework-tuned model averaged 10.0 (6.7%) residual vulnerable code snippets. Notably, the framework-tuned model produced the fewest security flaws before and after the iterative validation process.

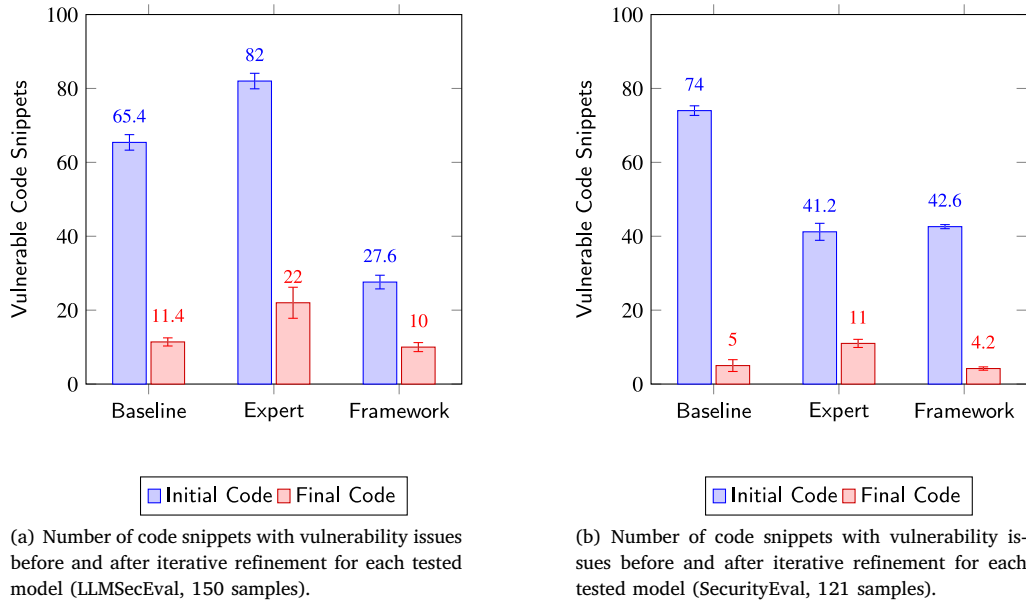


Fig. 7. Experiment #2: Comparison of the number of code snippets with vulnerability issues before and after iterative refinement for each tested model when using the LLMSEval (left) and SecurityEval (right) datasets; error bars indicate the standard deviation across five experiment runs.

A similar trend was observed with the SecurityEval dataset. Initially, the baseline GPT-4o, expert-tuned, and framework-tuned models produced an average of 74.0 (61.2%), 41.2 (34.1%), and 42.6 (35.2%) vulnerable code snippets, respectively, out of 121 total samples. After refinement, these numbers decreased to 5.0 (4.1%) for the baseline GPT-4o model, 11.0 (9.1%) for the expert-tuned model, and 4.2 (3.5%) for the framework-tuned model. Once again, the framework-tuned model produced the fewest residual vulnerable code snippets, demonstrating the self-refinement approach's effectiveness in minimizing security issues.

Fig. 8 shows, for each model, the cumulative percentage of code snippets that are free of vulnerabilities after each refinement iteration. At iteration 0, the percentage represents the proportion of secure code in the initial code generation. With each refinement, this percentage increases as more vulnerabilities are resolved, until it eventually plateaus when no further improvements can be made.

For example, the framework-fine-tuned model in SecurityEval starts with 64.9% secure code snippets at iteration 0 and quickly rises to 95.2% after one refinement. It reaches 96.2% by iteration 2 and remains stable thereafter. The baseline model begins with a lower proportion of secure code (38.6%), but it also reaches a similar final level (96.7%) after five iterations. The expert-fine-tuned model begins with 63.6% secure code snippets and gradually climbs to 92.0%. These results demonstrate that the framework-fine-tuned model generates more secure code from the beginning and achieves a high level of security with fewer refinement iterations than the other models. This highlights the efficiency and effectiveness of our framework.

Overall, the validation framework is an effective fine-tuning strategy that enables LLMs to generate more secure code with fewer iterations without relying on expert-written examples. Although the expert-tuned model was trained on curated data from security professionals, it required more revisions and left more vulnerabilities unresolved. In contrast, the framework-fine-tuned model learns from its own refinement process, proactively reducing vulnerabilities before validation.

This approach simplifies fine-tuning, making it a scalable, low-cost process that minimizes human involvement. The results suggest that

self-refinement via automated validation is a practical, competitive alternative to expert-guided fine-tuning. It achieves comparable or superior outcomes without manual dataset construction.

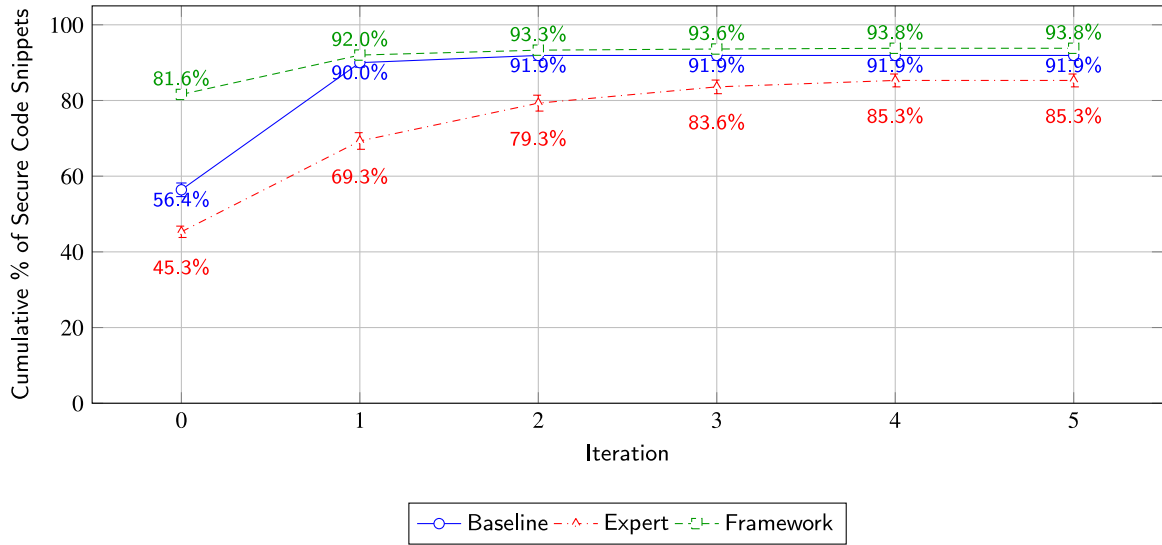
6.2.3. Functional correctness validation

Fig. 9 presents the number and percentage of code snippets with functional correctness issues at any point during the initial or refinement stages ("Initial/Refined Code"), as well as those with such issues in the final output ("Final Code"), for both LLMSEval and SecurityEval datasets, for each model.

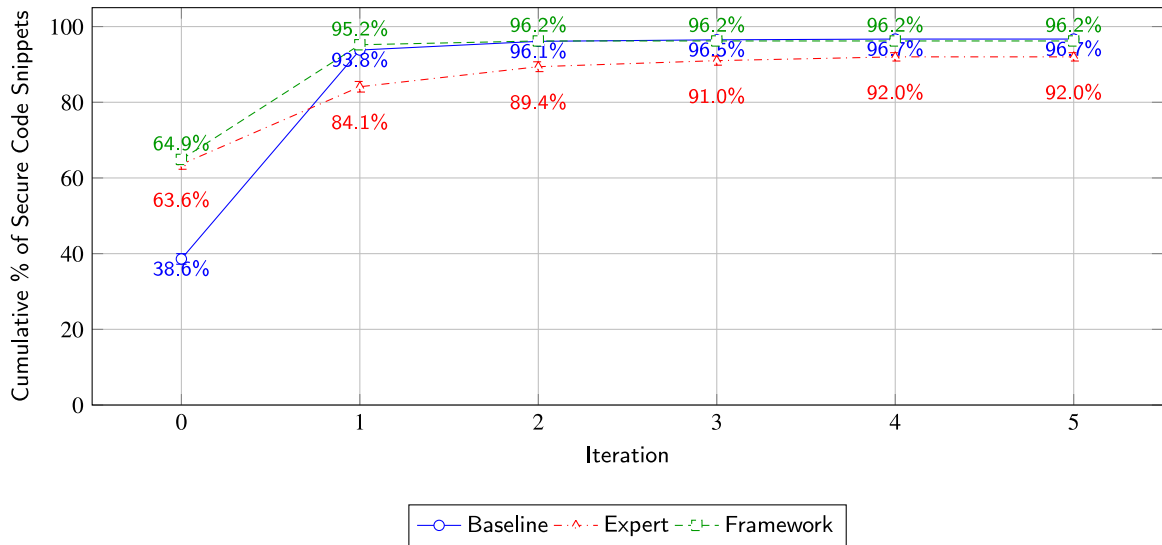
For LLMSEval (150 code snippets), the baseline model exhibited functional correctness errors in 8.2 cases (5.5%) during the initial or refinement stages, with only 1.2 cases (0.8%) remaining in the final output. The framework-tuned model triggered functional correctness errors in 21.8 cases (14.5%) during validation and refinement, however this number was decreased to just 0.4 cases (0.3%) in the final output, demonstrating strong self-correction capability. In contrast, the expert-tuned model exhibited a much higher number of functional correctness failures during validation and refinement (126.8 cases, 84.5%), and 118.4 cases (78.9%) still had errors in the final output, indicating little improvement.

Manual review revealed that the high number of incorrect code snippets for the expert-tuned model was largely due to its tendency to generate incomplete or partially implemented code. This resulted in frequent detection of functional errors throughout the validation process, with many issues persisting in the final output.

For LLMSEval, we found that 14 of the initial/refined code snippets generated by the framework-tuned model (9.3% of the total 150 samples) had functional correctness issues due to missing import statements for required libraries or modules. This issue frequently arose when the fine-tuned model employed distinct or more secure libraries than the GPT-4o model yet neglected to incorporate the requisite import statements. These cases represented a significant proportion of the overall functional correctness issues identified during the initial and refinement stages. Fortunately, these errors were easily resolved during the refinement process by automatically adding the missing import lines. We suspect that this pattern may be related to overfitting



(a) Cumulative percentage of secure code snippets after each refinement iteration (LLMSecEval, 150 samples).



(b) Cumulative percentage of secure code snippets after each refinement iteration (SecurityEval, 121 samples).

Fig. 8. Experiment #2: Cumulative percentage of secure code snippets generated after each refinement iteration; error bars indicate the standard deviation across five experiment runs, and labels show the average percentage for the corresponding iteration.

introduced during fine-tuning, but further investigation is needed to confirm the underlying cause.

A similar pattern was observed with SecurityEval (121 code snippets): the baseline model encountered functional correctness issues in 18.6 cases (15.4%) during the initial or refinement stages, with 4.2 (3.5%) remaining in the final output. The framework-tuned model had 17.6 cases (14.5%) with functional correctness errors during validation and refinement, but only 1.6 (1.3%) remaining in the final output. Once again, the expert-tuned model performed worst, with 108.2 incorrect cases during the initial or refined stages (89.5%) and 94.4 incorrect cases in the final output (78.0%).

These results highlight the framework-tuned model's effectiveness at resolving functional correctness issues at any stage of validation, not just in the initial code. In contrast, the expert-tuned model, fine-tuned on code outputs from prior LLMs, repeatedly generated incomplete or underspecified solutions. This resulted in persistent errors, even after several feedback cycles. Manual inspection suggests that, with validation-guided refinement, the framework-tuned model can reliably

address prompt requirements, quickly correcting errors as they arise and converging on correct outputs with fewer remaining failures.

7. Discussion

This section summarizes the main limitations of our framework and outlines directions for future work.

7.1. Limitations in functional correctness evaluation

Current metrics like CodeBLEU, CodeBERTScore, and ICE-Score mainly measure similarity to reference code, so they do not always capture true functional correctness. These metrics may penalize useful improvements that are not specified in the prompt. Thus, these scores do not accurately reflect the actual quality of LLM-generated code.

Our evaluation relies on static analysis and similarity rather than comprehensive test cases. While creating exhaustive tests would be more accurate, it is time-consuming and difficult to scale. Current methods also miss some semantic correctness issues, especially when

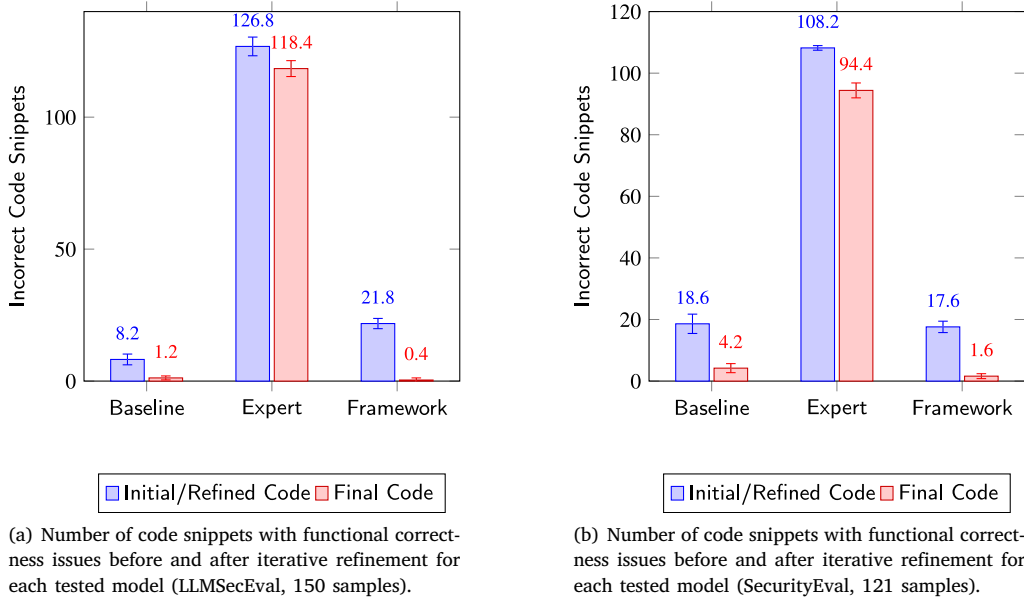


Fig. 9. Experiment #2: Comparison of the number of code snippets with functional correctness issues before and after iterative refinement for each tested model when using the LLMSEval (left) and SecurityEval (right) datasets; error bars indicate the standard deviation across five experiment runs.

there are multiple valid solutions. Advanced methods, such as formal verification, could help but are difficult to apply broadly. Improved evaluation approaches combining static analysis, dynamic testing, and semantic checks are needed.

7.2. Static analysis limitations

Our framework uses SAST tools to detect errors and security issues. This is an improvement over typical LLM-only approaches [26]. However, these tools do not always consider the entire code context, which can result in false positives and false negatives during analysis, as discussed in [40,41]. In practice, it is generally observed that static analysis tools can trigger unnecessary code modifications, while still missing certain vulnerabilities due to false positives and negatives. Previous research [42] has demonstrated that using multiple SAST tools can enhance detection accuracy and coverage, thereby reducing false positives and negatives. To address these issues, we plan to incorporate a broader range of SAST tools in the future.

7.3. Scalability beyond benchmarks and generalizability

The effectiveness of fine-tuning depends on the diversity and coverage of the training dataset. Although LLMSEval and SecurityEval provide a wide variety of security-related programming tasks, they do not fully represent all real-world or domain-specific scenarios. To ensure robustness as new security threats emerge, it will be essential to continuously expand the training and evaluation data with relevant, up-to-date examples. In addition, the availability of large, high-quality security datasets remains limited; therefore, we also plan to build more diverse, custom datasets for future LLM training.

Moreover, the current implementation and evaluation of CodeEnhancer are limited to standalone Python snippets. In practice, software systems are often substantially more complex than isolated code snippets and may involve larger codebases, richer dependencies, and other programming languages. As a result, the generalizability of our framework to more complex real-world projects and other languages remains unproven. Future work will focus on extending the framework to handle such complexity and language-specific integrations of SAST tools.

7.4. Fine-tuning data issues

As discussed in Section 6.2.3, the expert-tuned model exhibited substantially lower functional correctness and frequently generated incomplete implementations. We acknowledge that this outcome raises a potential concern about comparison bias: expert-written secure reference implementations may not always align with the structural conventions and output style that the target base model (GPT-4o) naturally tends to generate. Even when both solutions are correct and secure, they may differ in aspects such as library usage, abstraction level, and error-handling granularity.

This mismatch can be amplified during supervised fine-tuning and may lead to degraded behaviors, including incomplete or underspecified code generation, as observed in our experiments. While this represents a limitation in directly leveraging static expert corpora for fine-tuning rapidly evolving foundation models, it also motivates a practical strength of our approach. In CodeEnhancer, the framework-refined fine-tuning data is automatically derived from the target model's own outputs after iterative validation, producing training targets that are more consistent with the target model's preferred output format, while still satisfying the security and functional checks. As a result, the framework-refined data provides a scalable and adaptable fine-tuning pathway that reduces reliance on distribution-sensitive external reference implementations.

Nevertheless, we observed occasional behavioral anomalies in the intermediate outputs of the framework-tuned model that were introduced during fine-tuning. These anomalies included missing import statements, which compromised executability even when the intended logic was correct. These anomalies underscore the influence of fine-tuning data on generation behavior and suggest that, in practical deployments, fully autonomous code generation may still benefit from a lightweight human-in-the-loop review.

As a solution, in future work we will attempt to resolve these anomalies by integrating static dependency checks into the fine-tuning data preparation pipeline.

8. Conclusion

In this paper, we introduced CodeEnhancer, a unified framework for enhancing the security and quality of Python code generated by

LLMs. Our two-stage approach combines real-time validation with targeted fine-tuning to address syntax errors, security vulnerabilities, and functional correctness within a single pipeline.

The validation pipeline applied to the baseline GPT-4o model removed 82.8% of vulnerabilities and resolved all functional correctness issues in the first stage.

In the second stage, we fine-tuned the LLM using both expert-written and framework-refined code. The framework-tuned model achieved the lowest vulnerability rates consistently among all models. For instance, on the LLMSecEval dataset, the proportion of vulnerable code was decreased from 43.6% for baseline GPT-4o-generated code to 18.4% before refinement and further to 6.7% after refinement. On the SecurityEval dataset, vulnerability rates decreased from 61.2% for the GPT-4o baseline model to 35.2% before refinement and to 3.5% after refinement. Similar trends were observed for functional correctness.

Importantly, the framework-tuned model consistently outperformed the expert-tuned model in Experiment #2, achieving lower vulnerability rates and higher functional correctness on both benchmark datasets. These results highlight the effectiveness of automated self-refinement and suggest that high-quality self-generated data may be more beneficial for fine-tuning than relying solely on expert-written code.

Despite these improvements, limitations remain. Current metrics do not fully capture functional equivalence, static analysis cannot assess semantic correctness, and datasets may not cover all programming domains or new security threats. Addressing these challenges will require broader, continually updated datasets and better evaluation methods.

As future work, we plan to integrate more static and dynamic analysis tools, adopt advanced fine-tuning strategies, and test the framework in real-world development settings. These steps are necessary to increase coverage further, keep pace with evolving security practices, and better assess the practical reliability of AI-generated code.

As LLMs become central to software development workflows, generating secure and correct code is critical. Our proposed framework is a significant step toward that objective. It demonstrates that combining LLMs with static analysis and purpose-driven fine-tuning substantially increases the trustworthiness and reliability of AI-assisted coding systems.

List of abbreviations

API: Application Programming Interface.
AST: Abstract Syntax Tree.
Bandit: A security-oriented static analysis tool for Python.
CodeBERTScore: A metric for evaluating code via embedding-based semantic alignment.
CodeBLEU: A metric for evaluating code via n-gram, syntax, and data-flow similarity.
CodeQL: A semantic code analysis tool for vulnerability detection.
CodeScore: A metric for evaluating code via its predicted executability.
Codex: An early LLM for natural-language-to-code generation.
CWE: Common Weakness Enumeration, a taxonomy of software weaknesses.
GPT: Generative Pre-trained Transformer.
HumanEval: An execution-based benchmark for code functional correctness evaluation.
ICE-Score: An LLM-based instruction-compliance code evaluation metric.
LLM: Large Language Model.
LLMSecEval: A security-oriented benchmark dataset for code generation.
MBPP: Mostly Basic Programming Problems, a code generation benchmark.
Mitre: The organization managing the CWE taxonomy.
OWASP: Open Web Application Security Project.

PEFT: Parameter-Efficient Fine-Tuning.

PEP 8: The standard style guide for Python code.

Pylint: A Python static analysis and linting tool.

SAST: Static Application Security Testing.

SBC-Score: A metric for evaluating code via reverse-generation-based semantic comparison.

SecurityEval: A diverse security benchmark dataset used to evaluate the model's generalization capabilities beyond LLMSecEval.

SQL: Structured Query Language.

XML: Extensible Markup Language.

XSS: Cross-Site Scripting.

CRedit authorship contribution statement

Jongmin Lee: Writing – original draft, Software, Methodology, Investigation, Data curation, Conceptualization. **Khang Mai:** Methodology, Conceptualization. **Nakul Ghate:** Writing – review & editing, Methodology, Conceptualization. **Tomohiko Yagyu:** Supervision, Project administration, Funding acquisition. **Razvan Beuran:** Writing – review & editing, Supervision, Methodology, Funding acquisition, Conceptualization. **Yasuo Tan:** Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by JST SPRING, Japan Grant Number JPMJSP2102.

Data availability

I have shared the link to code at the manuscript.

References

- [1] OpenAI, GPT-4 technical report, 2023, arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774).
- [2] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X.E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C.C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, G. Synnaeve, Code Llama: Open foundation models for code, 2024, [arXiv:2308.12950](https://arxiv.org/abs/2308.12950), URL <https://arxiv.org/abs/2308.12950>.
- [3] M. Chen, J. Tworek, H. Jun, et al., Evaluating large language models trained on code, 2021, [arXiv:2107.03374](https://arxiv.org/abs/2107.03374), URL <https://arxiv.org/abs/2107.03374>.
- [4] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions, in: IEEE Symposium on Security and Privacy, SP, 2022, pp. 754–768.
- [5] N. Perry, M. Srivastava, D. Kumar, D. Boneh, Do users write more insecure code with AI assistants? in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 2785–2799, <http://dx.doi.org/10.1145/3576915.3623157>.
- [6] F. Tambon, A. Moradi-Dakheel, A. Nikanjam, F. Khomh, M.C. Desmarais, G. Antoniol, Bugs in large language models generated code: an empirical study, Empir. Softw. Engg. 30 (3) (2025) <http://dx.doi.org/10.1007/s10664-025-10614-4>.
- [7] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Examining GitHub Copilot's security vulnerabilities: A case study, IEEE Secur. Priv. 20 (5) (2022) 30–37.
- [8] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, S. Ma, CodeBLEU: A method for automatic evaluation of code synthesis, 2020, arXiv preprint [arXiv:2009.10297](https://arxiv.org/abs/2009.10297).
- [9] S. Zhou, D. Kang, F. Khani, A. Grover, S. Ren, CodeBERTScore: Evaluating code generation with pretrained models of code, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP, Association for Computational Linguistics, Singapore, 2023, pp. 13921–13937, <http://dx.doi.org/10.18653/v1/2023.emnlp-main.859>.

- [10] Y. Dong, J. Ding, X. Jiang, G. Li, Z. Li, Z. Jin, CodeScore: Evaluating code generation by learning code execution, *ACM Trans. Softw. Eng. Methodol.* 34 (3) (2025) <http://dx.doi.org/10.1145/3695991>.
- [11] PyCQA, Pylint, 2003, <https://github.com/PyCQA/pylint>.
- [12] PyCQA, Bandit, 2014, <https://github.com/PyCQA/bandit>.
- [13] C. Tony, M. Mutas, N.E.D. Ferreyra, R. Scandariato, LLMSEval: A dataset of natural language prompts for security evaluations, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR, IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 588–592, <http://dx.doi.org/10.1109/MSR59073.2023.00084>, URL <https://doi.ieeecomputersociety.org/10.1109/MSR59073.2023.00084>.
- [14] M.L. Siddiq, J.C.S. Santos, SecurityEval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques, in: Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security, MSR4P&S '22, Association for Computing Machinery, Singapore, Singapore, 2022, pp. 29–33, <http://dx.doi.org/10.1145/3549035.3561184>.
- [15] A.Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D.S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L.R. Lavaud, M.-A. Lachaux, P. Stock, T.L. Scao, T. Lavril, T. Wang, T. Lacroix, W.E. Sayed, Mistral 7B, 2023, [arXiv:2310.06825](https://arxiv.org/abs/2310.06825), URL <https://arxiv.org/abs/2310.06825>.
- [16] DeepSeek-AI, et al., DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025, [arXiv:2501.12948](https://arxiv.org/abs/2501.12948), URL <https://arxiv.org/abs/2501.12948>.
- [17] J. Wang, Y. Chen, A review on code generation with LLMs: Application and evaluation, in: 2023 IEEE International Conference on Medical Artificial Intelligence, MedAI, 2023, pp. 284–289, <http://dx.doi.org/10.1109/MedAI59581.2023.00044>.
- [18] B. Johnson, Y. Song, E. Murphy-Hill, R. Bowdidge, Why don't software developers use static analysis tools to find bugs? in: 2013 35th International Conference on Software Engineering, ICSE, 2013, pp. 672–681, <http://dx.doi.org/10.1109/ICSE.2013.6606613>.
- [19] GitHub, CodeQL, 2023, <https://codeql.github.com/>.
- [20] Y.-D. Tsai, M. Liu, H. Ren, RTLFixer: Automatically fixing RTL syntax errors with large language models, 2024, [arXiv:2311.16543](https://arxiv.org/abs/2311.16543), URL <https://arxiv.org/abs/2311.16543>.
- [21] A.Z. Nasrat, R. Sharma, S.K. Vadlamudi, I. Tahmid, L. Guerrouj, Unveiling inefficiencies in LLM-generated code: Toward a comprehensive taxonomy, 2025, [arXiv preprint arXiv:2503.06327](https://arxiv.org/abs/2503.06327).
- [22] J. Huang, X. Chen, S. Mishra, H.S. Zheng, A.W. Yu, X. Song, D. Zhou, Large language models cannot self-correct reasoning yet, 2024, [arXiv:2310.01798](https://arxiv.org/abs/2310.01798), URL <https://arxiv.org/abs/2310.01798>.
- [23] R. Kamoi, Y. Zhang, N. Zhang, J. Han, R. Zhang, When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs, "Transactions Comput. Linguistics" 12 (2024) 1417–1440, http://dx.doi.org/10.1162/tacl_a_00713, URL <https://aclanthology.org/2024.tacl-1.78/>.
- [24] J. Li, A. Sangalay, C. Cheng, Y. Tian, J. Yang, Fine tuning large language model for secure code generation, in: Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering, FORGE '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 86–90, <http://dx.doi.org/10.1145/3650105.3652299>.
- [25] M. Weyssow, X. Zhou, K. Kim, D. Lo, H. Sahraoui, Exploring parameter-efficient fine-tuning techniques for code generation with large language models, *ACM Trans. Softw. Eng. Methodol.* (2025) <http://dx.doi.org/10.1145/3714461>.
- [26] K. Alrashedy, A. Aljasser, P. Tambwekar, M. Gombolay, Can LLMs patch security issues? 2024, [arXiv:2312.00024](https://arxiv.org/abs/2312.00024), URL <https://arxiv.org/abs/2312.00024>.
- [27] H. Pearce, B. Tan, B. Ahmad, R. Karri, B. Dolan-Gavitt, Examining zero-shot vulnerability repair with large language models, in: IEEE Symposium on Security and Privacy, SP, 2023, pp. 1355–1369.
- [28] M. Keltek, R. Hu, M. Fani Sani, Z. Li, LSAST: Enhancing cybersecurity through LLM-supported static application security testing, 2024, [arXiv preprint arXiv:2409.15735](https://arxiv.org/abs/2409.15735), URL <https://arxiv.org/abs/2409.15735>.
- [29] T. Ridnik, D. Kredo, I. Friedman, Code generation with AlphaCodium: From prompt engineering to flow engineering, 2024, [arXiv:2401.08500](https://arxiv.org/abs/2401.08500).
- [30] T.X. Olsson, J.P. Inala, C. Wang, J. Gao, A. Solar-Lezama, Is self-repair a silver bullet for code generation? in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024, 2024, URL <https://openreview.net/forum?id=y0GJXRungR>.
- [31] A. Kumar, V. Zhuang, R. Agarwal, Y. Su, J.D. Co-Reyes, A. Singh, K. Baumli, S. Iqbal, C. Bishop, R. Roelofs, L.M. Zhang, K. McKinney, D. Shrivastava, C. Paduraru, G. Tucker, D. Precup, F.M.P. Behbahani, A. Faust, Training language models to self-correct via reinforcement learning, in: ICLR, OpenReview.net, 2025, URL <https://dblp.uni-trier.de/db/conf/iclr/iclr2025.html#KumarZASCSBIBRZ25>.
- [32] H. Hajipour, K. Hassler, T. Holz, L. Schonherr, M. Fritz, CodeLMsec benchmark: Systematically evaluating and finding security vulnerabilities in black-box code language models, in: 2024 IEEE Conference on Secure and Trustworthy Machine Learning, SaTML, IEEE Computer Society, Los Alamitos, CA, USA, 2024, pp. 684–709, <https://doi.org/10.1109/SaTML59370.2024.00040>, URL <https://doi.ieeecomputersociety.org/10.1109/SaTML59370.2024.00040>.
- [33] S.-C. Dai, J. Xu, G. Tao, A comprehensive study of LLM secure code generation, 2025, [arXiv:2503.15554](https://arxiv.org/abs/2503.15554), URL <https://arxiv.org/abs/2503.15554>.
- [34] SonarSource, SonarQube, 2007, <https://docs.sonarsource.com/>.
- [35] Checkmarx, Checkmarx SAST, 2006, <https://checkmarx.com/cxsast-source-code-scanning/>.
- [36] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, C. Sutton, Program synthesis with large language models, 2021, [arXiv:2108.07732](https://arxiv.org/abs/2108.07732), URL <https://arxiv.org/abs/2108.07732>.
- [37] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, S.K. Lahiri, LLM-based test-driven interactive code generation: User study and empirical evaluation, *IEEE Trans. Softw. Eng.* 50 (9) (2024) 2254–2268, <http://dx.doi.org/10.1109/TSE.2024.3428972>.
- [38] T.Y. Zhang, Z. Wang, Y. Wang, K.-H. Chan, J.C.K. Cheung, ICE-Score: Instructing large language models to evaluate code, in: Findings of the Association for Computational Linguistics: EACL 2024, Association for Computational Linguistics, St. Julian's, Malta, 2024, pp. 2141–2153.
- [39] A.Z. Nasrat, R. Sharma, S.K. Vadlamudi, I. Tahmid, L. Guerrouj, Bridging LLM-generated code and requirements: Reverse generation technique and SBC metric for developer insights, 2025, [arXiv preprint arXiv:2502.07835](https://arxiv.org/abs/2502.07835).
- [40] J. Ruohonen, K. Hjerpe, K. Rindell, A Large-Scale Security-Oriented Static Analysis of Python Packages in PyPI, in: 2021 18th International Conference on Privacy, Security and Trust, PST, IEEE Computer Society, Los Alamitos, CA, USA, 2021, pp. 1–10, <http://dx.doi.org/10.1109/PST52912.2021.9647791>, URL <https://doi.ieeecomputersociety.org/10.1109/PST52912.2021.9647791>.
- [41] A. Kaur, R. Nayyar, A comparative study of static code analysis tools for vulnerability detection in C/C++ and JAVA source code, *Procedia Comput. Sci.* 171 (2020) 2023–2029, <http://dx.doi.org/10.1016/j.procs.2020.04.217>, URL <https://www.sciencedirect.com/science/article/pii/S1877050920312023>, Third International Conference on Computing and Network Communications (CoCoNet'19).
- [42] A. Nguyen-Duc, M.V. Do, Q. Luong Hong, K. Nguyen Khac, A. Nguyen Quang, On the adoption of static analysis for software security assessment—A case study of an open-source e-government project, *Comput. Secur.* 111 (2021) 102470, <http://dx.doi.org/10.1016/j.cose.2021.102470>, URL <https://www.sciencedirect.com/science/article/pii/S0167404821002947>.